SPE-229218-MS

Rust-Powered Engine for Accelerated SEG-Y File Slicing

A. S. Dubovik, R. L Khudorozhkov, A. A. Kozhevin, R. Garipov, A. Y. Altynova, and D. Novikov, WAIW LLC, Dubai, Dubai, UAE; N. Suurmeyer and T. Jones Martin, ThinkOnward LLC, Austin, TX, USA

Copyright 2025, Society of Petroleum Engineers DOI 10.2118/229218-MS

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Seismic data processing and interpretation are central to modern exploration workflows, where efficient parsing and visualization of SEG-Y files directly impacts operational success [8, 9]. The extraction of inline and crossline sections from three-dimensional seismic post-stack cubes enables identification of subtle subsurface features including faults, horizons, facies, and potential hydrocarbon traps. Current processing solutions struggle with modern seismic datasets, particularly as real-time interpretation and machine learning applications demand sustained high-throughput data delivery to maintain computational efficiency, all the while existing processing tools frequently exhibit throughput limitations that constrain these advanced analytical approaches [10].

Traditional implementations like segyio, while widely adopted, struggle particularly with crossline slicing operations where data must be gathered from non-contiguous file locations [1]. The segfast library with memmap engine attempts to address these issues through memory-mapped file handling but encounters inconsistent performance across different scenarios [2]. OpenVDS offers a brick- based volumetric format that can achieve superior performance in specific cases, but requires scanning and importing SEG-Y files into a proprietary data store before optimal access is possible, creating preprocessing overhead and workflow integration challenges [3]. The Rust programming language combines memory safety, performance optimization, and concurrency capabilities that address these specific challenges [4, 12, 13]. Unlike C-based implementations that risk memory management errors, Rust's ownership model provides guaranteed memory safety without performance penalties [14]. The language's zero-cost abstractions enable fine-grained optimization of critical processing paths, while sophisticated concurrency primitives allow efficient utilization of multi-core processing environments common in modern interpretation workstations.

This research presents a Rust-based engine for accelerating SEG-Y file slicing through optimized memory management, buffer allocation, and concurrent processing. We benchmark performance across different access patterns, dataset sizes, and storage configurations, comparing against segyio, segfast with memmap, and OpenVDS using controlled testing with statistical validation. The Rust-based approach achieves extraction speeds up to 4x faster than traditional implementations, with the most pronounced gains in random crossline access operations. Unlike format-conversion approaches, this implementation reads SEG-Y files directly without preprocessing, allowing immediate integration into existing workflows.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

Introduction

Seismic data processing and interpretation are central to modern exploration workflows, where efficient parsing and visualization of SEG-Y files directly impacts operational success [8, 9]. The extraction of inline and crossline sections from three-dimensional seismic post-stack cubes enables identification of subtle subsurface features including faults, horizons, facies, and potential hydrocarbon traps. Current processing solutions struggle with modern seismic datasets, particularly as real-time interpretation and machine learning applications demand sustained high-throughput data delivery to maintain computational efficiency, all the while existing processing tools frequently exhibit throughput limitations that constrain these advanced analytical approaches [10].

Traditional implementations like segyio, while widely adopted, struggle particularly with crossline slicing operations where data must be gathered from non-contiguous file locations [1]. The segfast library with memmap engine attempts to address these issues through memory-mapped file handling but encounters inconsistent performance across different scenarios [2]. OpenVDS offers a brick- based volumetric format that can achieve superior performance in specific cases, but requires scanning and importing SEG-Y files into a proprietary data store before optimal access is possible, creating preprocessing overhead and workflow integration challenges [3]. The Rust programming language combines memory safety, performance optimization, and concurrency capabilities that address these specific challenges [4, 12, 13]. Unlike C-based implementations that risk memory management errors, Rust's ownership model provides guaranteed memory safety without performance penalties [14]. The language's zero-cost abstractions enable fine-grained optimization of critical processing paths, while sophisticated concurrency primitives allow efficient utilization of multi-core processing environments common in modern interpretation workstations.

This research presents a Rust-based engine for accelerating SEG-Y file slicing through optimized memory management, buffer allocation, and concurrent processing. We benchmark performance across different access patterns, dataset sizes, and storage configurations, comparing against segyio, segfast with memmap, and OpenVDS using controlled testing with statistical validation. The Rust-based approach achieves extraction speeds up to 4x faster than traditional implementations, with the most pronounced gains in random crossline access operations. Unlike format-conversion approaches, this implementation reads SEG-Y files directly without preprocessing, allowing immediate integration into existing workflows.

Methodology

The Rust-based SEG-Y slicing engine exploits three key language features to overcome traditional implementation limitations: 1) memory safety without runtime overhead, 2) compile-time concurrency guarantees, and 3) zero-cost abstractions. Rust's ownership model ensures memory safety without performance cost associated with automatic memory management [5]. Buffer allocations occur deterministically, preventing the memory leaks and buffer overflows that degrade C-based implementations during long interpretation sessions. These language-level guarantees enable aggressive optimizations—such as lock-free concurrent trace processing—that would be unsafe in traditional implementations. Rust's type system prevents data races at compile time, eliminating the runtime synchronization overhead of traditional locking mechanisms. This allows parallel processing of trace data across multiple cores without thread safety risks.

Crossline access operations present particular challenges. While inline slicing benefits from sequential access patterns where traces are stored contiguously, crossline operations require gathering data from scattered file locations —each seek jumping across (number_of_inlines * trace_size) bytes. The implementation addresses this through predictive buffer pre-fetching and concurrent read operations. Native SEG-Y parsing handles various encoding formats without the preprocessing required by solutions like OpenVDS, enabling immediate integration into existing workflows [6].

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

To establish the practical benefits of this Rust-based approach, a comprehensive benchmarking study was designed to quantify performance improvements across diverse operational scenarios. The evaluation methodology includes comparison with established processing solutions under controlled conditions, ensuring statistically valid measurements that reflect real-world performance characteristics.

Benchmarking Methodology

When benchmarking SEG-Y libraries, several challenges arise. Caching can significantly skew results, as both operating system and hardware caching may artificially boost performance. Memory layout and access patterns are critical—contiguous versus random access markedly affects performance, particularly for slicing operations. Inline and crossline operations exhibit fundamentally different I/O characteristics. We addressed these challenges through multiple benchmark runs, flushing the OS cache between each execution to isolate true processing performance. Each scenario was run thirty times, calculating the mean of each run and then the grand mean to ensure statistical validity. The fundamental measurement unit is time required to acquire individual slices, with scenarios covering different slice counts, directions (inline or crossline), and access patterns (sequential or random). We compared against three established SEG-Y processing solutions: segfast with segyio engine (traditional C-based implementation), segfast with memmap engine (memory- mapped file handling), and OpenVDS (brick-based volumetric format) [1, 2, 3]. OpenVDS requires scanning and importing SEG-Y files into a volume data store before optimal access —a preprocessing overhead that applies only to initial access, as the converted format excels once cached in memory.

Benchmarking was conducted on Amazon EC2 c6a.xlarge instances, providing standardized computational resources with 4 vCPUs and 8 GiB of memory running Ubuntu 24.04.1 LTS and Python 3.11. This configuration represents typical interpretation workstation capabilities while ensuring consistent hardware characteristics across all testing scenarios. Storage configuration testing utilized Amazon Elastic Block Store with two distinct performance profiles to evaluate I/O scaling characteristics. Standard performance evaluation employed gp3 SSD storage providing 3,000 IOPS baseline performance, representative of typical interpretation workstation storage configurations. High-performance testing utilized io1 SSD storage delivering 16,000 IOPS capacity, enabling assessment of performance scaling potential under optimal storage conditions. The dual storage configuration approach enables determination of whether performance limitations originate from library I/O efficiency or hardware capabilities. Different engines may report similar times while varying in how efficiently they utilize available storage bandwidth.

Testing used two post-stack SEG-Y files of different sizes: Vincent (1.2 GB, 309 inlines, 1126 crosslines, 1150 samples per trace) and B-26-92-TX (7.8 GB, 418 inlines, 2033 crosslines, 2250 samples per trace). Both volumes arrange traces by inline and use big- endian IEEE float encoding. The size difference reveals performance scaling characteristics—Vincent establishes baseline performance while B-26-92-TX demonstrates behavior under realistic operational loads.

Our testing covered the following scenarios: edge slices (first and last inline/crossline) for baseline performance; multiple slice operations (five slices, sequential and random) for batch processing; and cached versus uncached performance to reflect real-world usage. Cached testing involved preloading slices before measurement, while uncached scenarios flushed caches between runs. Random access patterns simulate interactive interpretation and multi-user environments. These scenarios specifically address the fundamental difference between inline and crossline operations: inline slicing benefits from sequential disk access with contiguous traces, while crossline slicing requires random access to gather scattered data throughout the file—a critical distinction for performance evaluation.

Each scenario underwent thirty independent runs with complete cache flushing between iterations. Individual run means were aggregated to calculate grand means, mitigating transient system variations. The thirty-run sample size provides sufficient statistical power to detect meaningful performance differences in I/O-intensive operations. Timing measurements were rounded to millisecond precision, except when

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

rounding would eliminate meaningful differences between engines. This comprehensive methodology establishes a firm foundation for evaluating the Rust-based engine against established solutions, providing quantitative evidence for performance improvements across diverse operational scenarios while maintaining strict scientific validity.

Evaluation and Results

Our benchmarking study shows the Rust-based engine outperforming traditional implementations across most testing scenarios. Results vary significantly with operational conditions—caching, storage speed, and access patterns all affect the performance hierarchy. The evaluation identifies where each engine excels and where it struggles, with some surprising reversals in cached scenarios that challenge initial assumptions about format efficiency. Improvements scale with operational complexity, showing the most dramatic gains in challenging scenarios such as random crossline access on large datasets. As an example, Figure 1 illustrates the relative performance differences for one of the most demanding scenarios: random crossline slicing on the larger B-26-92-TX dataset. The results also reveal important insights about caching behavior and storage scaling that influence real-world deployment considerations. These findings establish new benchmarks for SEG-Y processing efficiency while showing the practical benefits of modern programming approaches for geoscience applications.

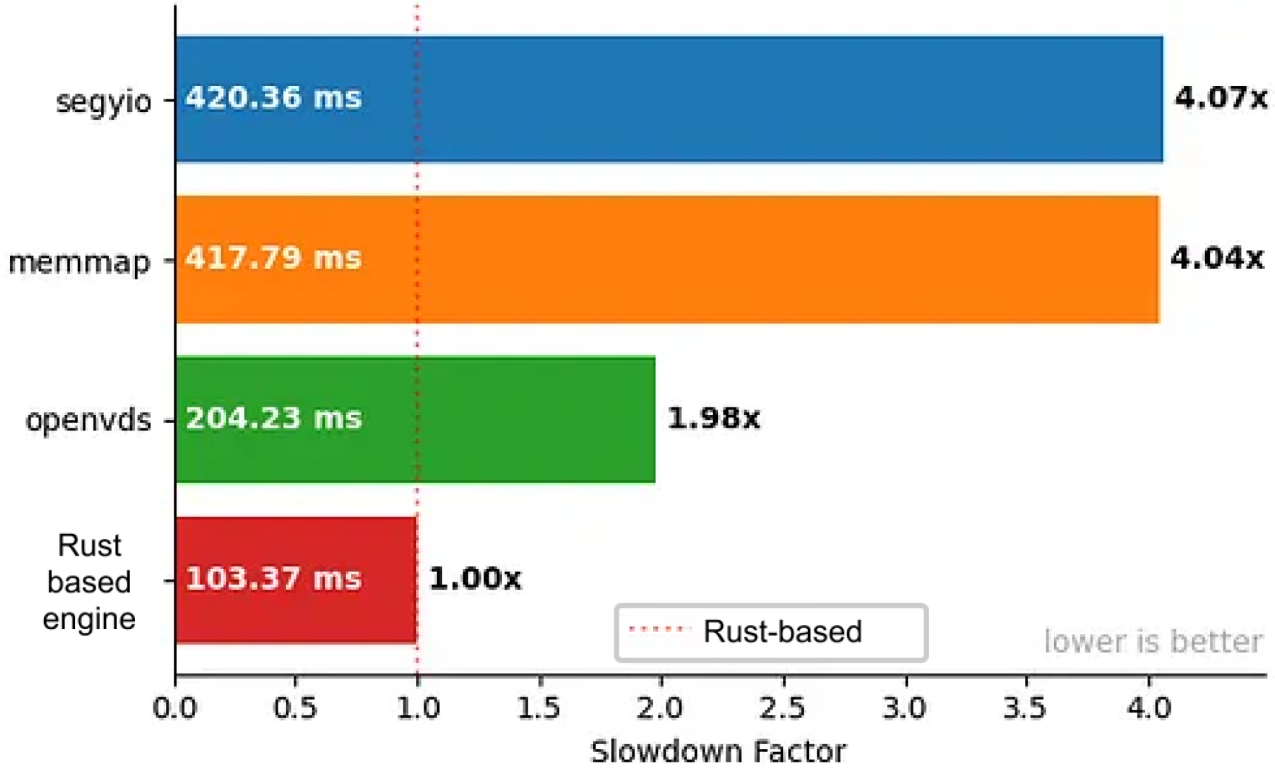

Figure 1—Relative slowdown for 5 random crossline slices in B-26-92-TX. Higher bars indicate slower performance relative to the Rust-based engine.

Single Slice Performance

Edge slice testing—first and last inline/crossline extractions—reveals baseline performance characteristics. The Rust engine runs 1.6 to 2.0 times faster than segyio across these basic operations, with the advantage growing to 4x for challenging cases like last crossline extraction on larger datasets. First inline slice extraction shows the baseline advantage, with the Rust engine achieving at least 1.6 to 2.0 times faster execution compared to segyio across both test datasets, as detailed in Table 1. The memmap engine typically

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

matches segyio performance but shows puzzling anomalies—sometimes taking over one second for the first slice on Vincent, yet performing normally on the larger B-26-92-TX dataset. This inconsistency appeared repeatedly in our tests but lacks a clear explanation.

Table 1—Grand mean time in milliseconds of extracting the first inline slice. Green highlight shows the quickest engine for a file-storage combination.

engine	Vincent		B-26-92-TX	
	3'000 iops	16'000 iops	3'000 iops	16'000 iops
segyio	24	24	123	106
memmap	1137	1134	123	108
openvds	936	276	7915	1196
Rust-based engine	15	14	61	54
Rust speedup	1.58x	1.76x	2.03x	1.95x

Figure 2 demonstrates the relative performance differences for first inline slice operations across both datasets. OpenVDS exhibits substantially different characteristics, requiring 20 to 130 times longer than the Rust engine for first slice operations. However, OpenVDS shows notable improvement with high-throughput storage configurations, displaying greater sensitivity to I/O capabilities than competing solutions. This preprocessing penalty applies only to initial access; the converted brick-based format excels once cached in memory.

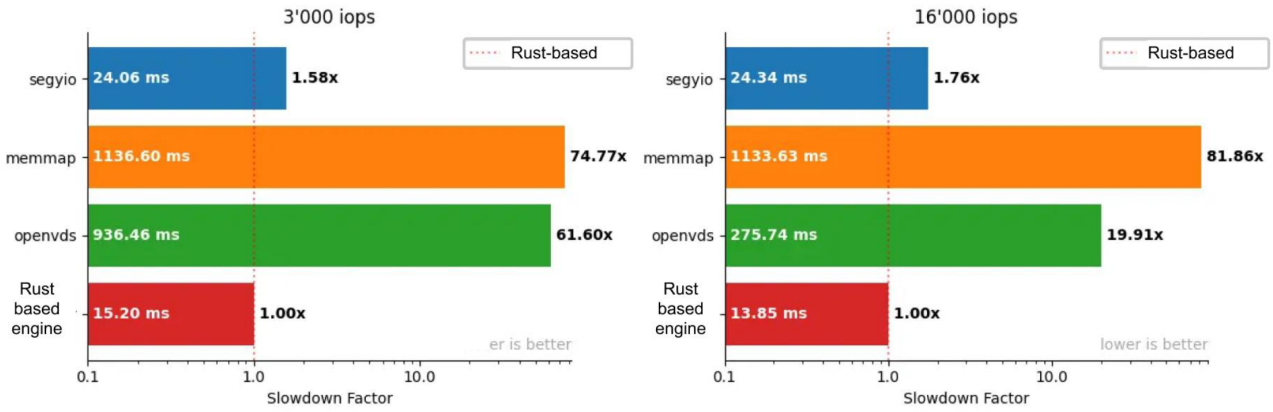

Figure 2—Relative slowdown for first inline slice operations.

Crossline edge slice operations reveal more pronounced differentials, particularly for last crossline extraction. On the smaller Vincent dataset, differences between the Rust engine and segyio remain modest. However, the larger B-26-92-TX dataset shows dramatic scaling, with the Rust engine achieving up to 4.0 times faster extraction compared to segyio and memmap engines, as illustrated in Figure 3. The scaling with dataset size indicates that Rust-based optimizations become increasingly effective as data volume increases. OpenVDS efficiency improves substantially on larger datasets with high-throughput storage, achieving second-place results with approximately 1.4 times slower execution than the Rust engine, while segyio and memmap engines show 4.0 times slower execution on the same scenarios.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

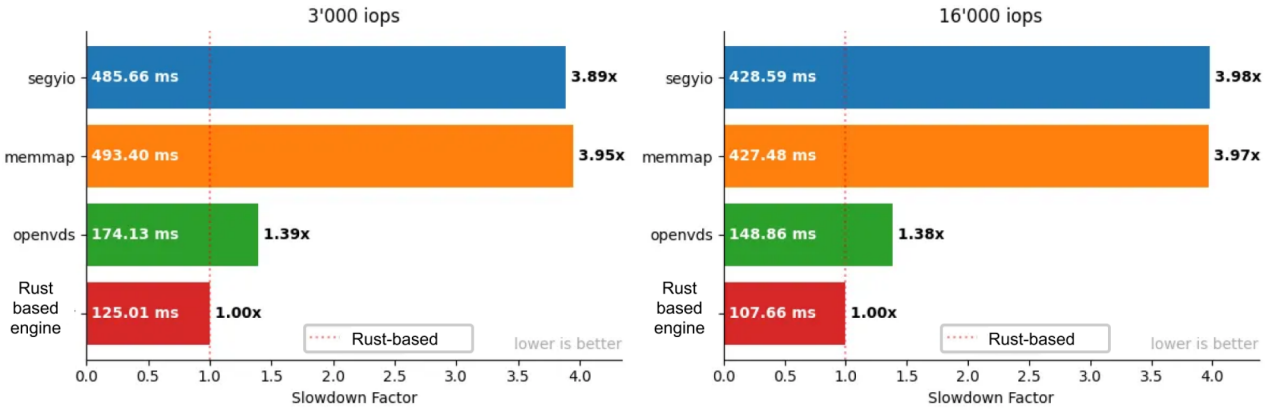

Figure 3—Relative slowdown for the last crossline on B-26-92-TX. Multiple Slice Performance

Multiple Slice Performance

Sequential and random access patterns for multiple slice operations show consistent advantages across varying operational scenarios. Five-slice extraction testing reveals that the Rust engine maintains improvements in the 1.6 to 2.3 times range compared to traditional implementations, with characteristics largely independent of slice spatial location for inline operations. Sequential inline slicing maintains advantages similar to single slice operations, indicating that batch processing overhead remains minimal. The consistency of processing times regardless of slice position within the volume indicates efficient buffer management and memory allocation strategies. OpenVDS approaches competitive efficiency in sequential scenarios on high-throughput storage, achieving approximately 5.0 times slower execution compared to the Rust engine on both test datasets. Random inline slicing preserves similar characteristics to sequential operations, suggesting that optimizations remain effective even when slice access patterns cannot be predicted. This consistency proves particularly valuable for interactive interpretation scenarios where user navigation patterns cannot be anticipated, and for machine learning applications where training data batches may require random sampling across seismic volumes.

Crossline operations exhibit more complex patterns that reflect the fundamental I/O challenges associated with gathering scattered trace data. Sequential crossline operations show improvements similar to inline scenarios on smaller datasets, with segyio approaching Rust engine efficiency on the Vincent volume. Larger datasets reveal more substantial gaps, with the Rust engine maintaining clear advantages even in sequential access patterns. Random crossline operations present the most challenging scenario, where processing times increase substantially compared to sequential operations across all tested engines. The Rust engine maintains significant advantages under these demanding conditions, with improvements that scale with dataset size and storage configurations. High- performance storage enables the most dramatic gains, with the Rust engine showing up to 4.1 times faster execution for random crossline operations on large datasets, as demonstrated in Figure 4.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

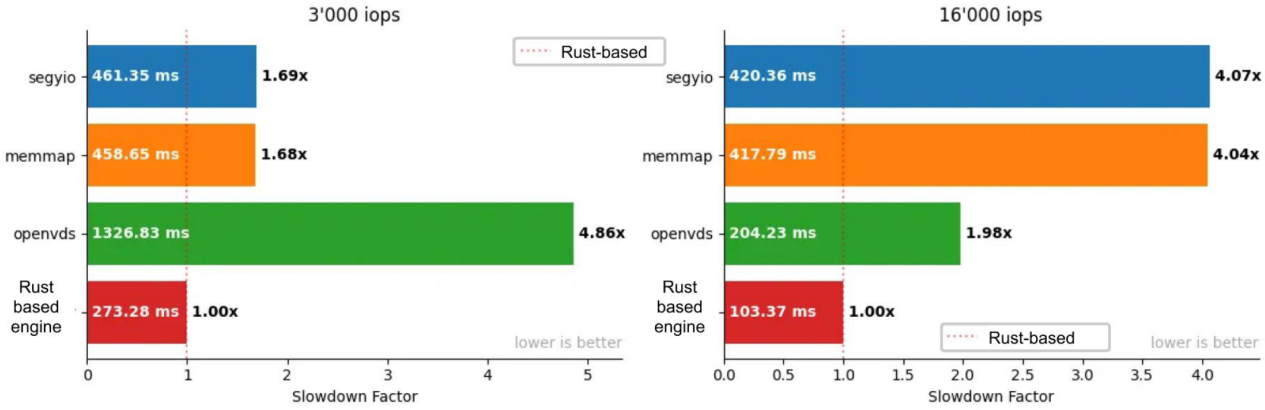

Figure 4—Performance comparison for 5 crossline random slices.

Caching Effects

Caching changes everything: OpenVDS suddenly becomes the fastest engine for sequential cached operations, beating even our Rust implementation. When data resides in RAM, I/O optimizations become irrelevant and OpenVDS's brick-based format—a liability for disk access—becomes an asset for memory traversal. This reversal only applies to sequential access; random cached operations still favor the Rust engine. Sequential cached operations show notable shifts in relative engine efficiency. For cached inline sequential slicing, OpenVDS emerges as the faster solution, outperforming the Rust engine across both test datasets, as shown in Figure 5. This reversal highlights the effectiveness of the brick-based format design for sequential access patterns when I/O bottlenecks are eliminated through caching. The volumetric data organization provides clear advantages for contiguous data retrieval from memory-resident files. OpenVDS maintains its sequential cached advantages across crossline operations as well, achieving fastest execution times for cached crossline sequential scenarios. This consistent pattern across both slice orientations indicates that the volumetric format's organizational benefits become most apparent when storage latency is removed, allowing the format's inherent design advantages to dominate execution speed.

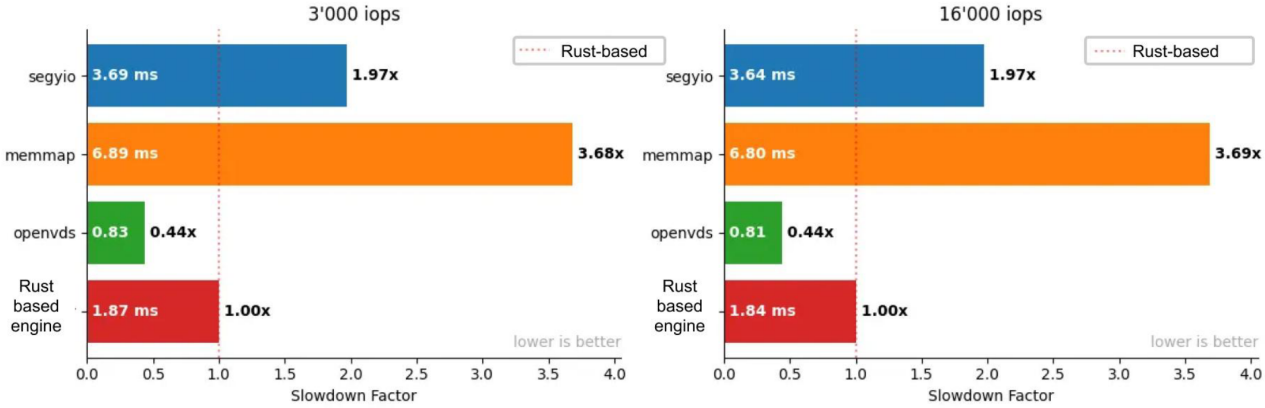

Figure 5—Relative slowdown for 10 inline sequential cached slices. OpenVDS comes forward as the faster solution when files are cached in memory, demonstrating the advantages of brick-based format design.

However, cached random access reveals different competitive patterns. Random cached operations show the Rust engine maintaining competitive positioning relative to segyio and memmap engines, while OpenVDS exhibits considerable slowdowns compared to its sequential cached efficiency, as illustrated in Figure 6. Random cached inline operations yield approximately equal execution times between the Rust engine, segyio, and memmap, while OpenVDS shows significantly slower results, particularly on high-performance storage. The storage independence of cached operations is evident in minimal differences

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

between 3,000 IOPS and 16,000 IOPS configurations for most engines. When data resides in memory, storage hardware becomes irrelevant for the Rust engine, segyio, and memmap solutions. OpenVDS continues to show some storage sensitivity even in cached scenarios, suggesting that its implementation retains dependency on storage beyond pure memory access patterns.

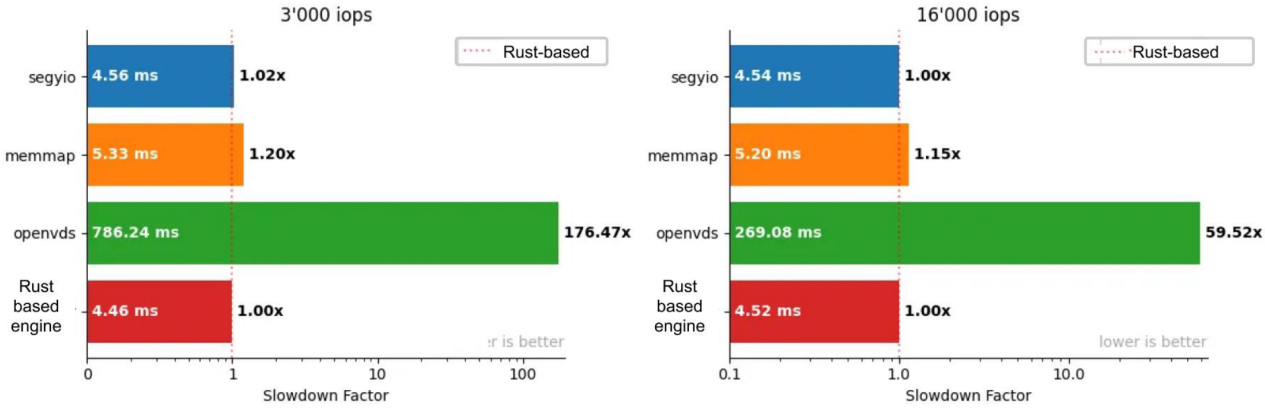

Figure 6—Relative slowdown for 10 inline random cached slices showing that OpenVDS performance degrades significantly in random access scenarios while other engines maintain similar efficiency.

Storage Impact

Faster storage dramatically improves Rust engine performance: crossline random operations on B-26-92-TX run 4.1x faster than segyio on 16,000 IOPS storage versus only 2.8x on 3,000 IOPS storage. Traditional engines barely benefit from the faster drives—they appear CPU-bound rather than I/O-bound—while OpenVDS shows the most sensitivity, sometimes improving by an order of magnitude. Crossline random operations exhibit the strongest storage scaling. On the larger B-26-92-TX dataset, high-performance storage enables the Rust engine to achieve 4.1 times faster execution compared to segyio, representing substantial improvement over the same scenario on standard storage. This scaling proves the Rust engine isn't just algorithmically faster—it actually generates more concurrent I/O operations, saturating fast storage while traditional engines leave that bandwidth unused.

OpenVDS shows the most pronounced storage sensitivity across all tested scenarios. While OpenVDS yields poor results on standard storage configurations, high-performance storage substantially improves its competitive positioning. The engine benefits significantly from increased IOPS capacity, suggesting that its brick-based format approach requires considerable I/O bandwidth to achieve optimal efficiency. This storage dependency represents a key consideration for deployment scenarios where storage configurations may vary.

Sequential operations show more modest storage scaling effects compared to random access patterns. The Rust engine maintains consistent advantages across both storage configurations for sequential scenarios, indicating that these access patterns do not fully utilize available I/O bandwidth improvements. This suggests that processing optimizations rather than pure I/O throughput determine execution speeds for sequential slice extraction. Storage scaling analysis also reveals that traditional engines like segyio and memmap show limited improvement with high-performance storage configurations. These engines appear limited by single-threaded parsing operations that cannot generate sufficient I/O requests to saturate faster storage.

Conclusion

This benchmarking study proves that a Rust-based approach can accelerate SEG-Y file slicing by up to 4x compared to traditional implementations. The gains are largest where they matter most—random

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

crossline access on large datasets, the bane of seismic interpretation workflows. Beyond raw speed, the results demonstrate that language choice and implementation strategy fundamentally affect how well software utilizes modern hardware capabilities. The Rust implementation succeeds not through incremental optimization but by exploiting language features unavailable in C: guaranteed memory safety enables aggressive parallelization, while zero-cost abstractions allow high-level code without performance penalties. The result is software that runs faster while being demonstrably safer due to Rust's compile-time memory guarantees and more maintainable due to its modern type system. In practice, this means interpretation workstations with 8 or 16 cores can finally use all that processing power for SEG-Y operations, while eliminating the memory leaks that force periodic restarts of long-running interpretation sessions—problems that have plagued C-based implementations for decades.

The practical advantages extend beyond speed. Unlike OpenVDS, which requires converting entire SEG-Y files before use, the Rust engine reads files directly—no preprocessing, no duplicate storage, no workflow disruption. Teams can point it at their existing data and immediately see performance improvements. This drop-in compatibility matters: the difference between a tool that gets adopted and one that remains a benchmark curiosity. Our benchmarking methodology—thirty runs per scenario, systematic cache flushing, dual storage configurations—revealed surprising truths about SEG-Y processing. Traditional engines like segyio cannot saturate modern SSDs, leaving performance on the table. OpenVDS swings from worst to first depending on caching. These insights emerged only through rigorous testing that isolated variables most benchmarks conflate.

For interpreters, 4x faster crossline access means fluid navigation through seismic volumes—no more coffee breaks while waiting for sections to load. For machine learning and AI workflows[7, 11], faster random access enables larger training batches and more data augmentation without GPU starvation. The improvements are consistent enough that teams can count on them: whether scanning for faults, tracking horizons, or training neural networks, the acceleration remains reliable. Machine learning workflows see transformative gains: preparing a 10,000-slice training dataset that previously took an hour now completes in 15 minutes. Random sampling for data augmentation—essential for robust models—no longer bottlenecks GPU utilization. These improvements compound: faster iteration enables more experiments, leading to better models in less time. The substantial speedup for random crossline operations on large datasets particularly benefits batch processing scenarios common in neural network training pipelines.

Workflows focused on structural analysis, such as fault identification and horizon tracking, can leverage the consistent crossline processing improvements. These interpretation tasks typically involve thorough examination of crossline sections to understand subsurface geology, and faster slice extraction enables more thorough analysis within existing project schedules. The scaling benefits observed on larger datasets suggest that complex interpretation projects may see proportionally greater workflow improvements. The direct SEG-Y compatibility offers immediate integration advantages for organizations seeking to improve their processing capabilities. Unlike solutions requiring preprocessing steps, this approach allows teams to realize processing improvements without modifying established workflows or investing in format conversion infrastructure. This compatibility reduces technical barriers and implementation complexity that often accompany the adoption of new tools.

Looking toward future developments, the established performance characteristics suggest several promising research directions. Advanced caching strategies could further optimize repeated access patterns common in interpretation workflows. The scaling benefits observed with high-performance storage indicate potential for GPU acceleration approaches that could extend these improvements to even more demanding computational scenarios. Cloud deployment looks particularly promising: the engine's ability to exploit high- IOPS storage aligns perfectly with cloud providers' high-performance storage tiers. Teams could run interpretation workloads on cloud instances with network-attached storage, paying for speed only when needed. The consistent performance across configurations means predictable costs—critical for cloud budget planning. As seismic datasets grow larger and interpretation teams more distributed, the

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025

ability to efficiently access SEG-Y data becomes even more critical. This work proves that order-of-magnitude improvements are possible—not through exotic hardware or formats, but through better software engineering.

References

1. Equinor ASA, "*segyio: Fast Python library for SEGY files*," GitHub Repository, 2025. Available: https://github.com/equinor/segyio
2. R. Khudorozhkov et al, "*segfast: Interacting with SEG-Y storage of seismic data*," GitHub Repository, 2025. Available: https://github.com/analysiscenter/segfast
3. The Open Group / Bluware, Inc., "Volume Data Store (VDS) Specification," *OpenVDS 3.4.255 documentation*, 2025.
4. W. Veytsman, J. Cashman, G. Escolona, and Y. Wu, "Rewrite it in Rust: A Computational Physics Case Study," arXiv preprint arXiv:2410.19146, 2024.
5. K. Dahiya and A. Dharani, "Building High-Performance Rust Applications: A Focus on Memory Efficiency," *SSRN Electronic Journal*, 2023.
6. K. M. Barry, D. A. Cavers, and C. W. Kneale, "Recommended standards for digital tape formats," *Geophysics*, vol. **40**, no. 2, pp. 344–352, 1975.
7. C. Birnie, M. Ravasi, and T. Alkhalifah, "Machine learning for seismic exploration: Where are we and how far are we from the holy grail?" *Geophysics*, vol. **89**, no. 1, pp. WA157–WA176, 2024.
8. J. R. Arrowsmith, et al, "Big Data Seismology," *Reviews of Geophysics*, vol. **60**, e2021RG000769, 2022.
9. M. A. Fisher, et al, "ExSeisDat: A set of parallel I/O and workflow libraries for petroleum seismology," *Oil & Gas Science and Technology*, vol. **73**, p. 73, 2018.
10. J. Mohan and V. Chidambaram, "The New Bottlenecks of ML Training: A Storage Perspective," *SIGARCH Computer Architecture News*, 2021.
11. A. Dubovik, A. Altynova, R. Khudorozhkov, A. Kozhevin, D. Novikov, N. Leseur, N. Suurmeyer, and T. Martin, "Innovative AI Workflow for Seismic Interpretation," SPE Conference Paper, SPE-224564-MS, 2025.
12. Nature Editorial Team, "Why scientists are turning to Rust," *Nature*, vol. **588**, pp. 185–186, 2020.
13. W. Bugden and A. Alahmar, "Rust: The Programming Language for Safety and Performance," arXiv preprint arXiv:2206.05503, 2022.
14. H. Xu, et al, "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs," arXiv preprint arXiv:2003.03296, 2020.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 08 November 2025